

CAPSTONE PROJECT REPORT

DEVSECOPS

Cloud-Based Secure Web Application

SUBMITTED BY

Kiran S

IPSR Solution Pvt Ltd

AWS – Alex Andrews

2026 - 11 - 06

Contents

PHASE 1. Planning and Setup 3
Step 1. Project Plan and Architecture 3
Step 2. Architecture Diagram and Explanation 4
Step 3. Create the GitHub repository 5
Step 4. Set up the local development environment 6
Step 5. Create the first Flask page 6
Step 6. Merge changes using a branch and pull request 6
PHASE 2. Application Development 7
Step 1. Create the Database and User model 7
Step 2. Build user registration with password hashing 7
Step 3. Add login, logout, and sessions 8
Step 4. Add tickets: create, list, and view 8
Step 5. Add the admin role, dashboard, and admin actions 9
Step 6. Add rate limiting and brute-force protection 9
Step 7. Add security headers and production settings 10
Step 8. Design the user interface 11
PHASE 3. AWS Networking 11
Step 1. Account Safety 11
Step 2. Build the network (VPC, subnets, and routing) 11
Step 3. Creating the security group 12
PHASE 4. Database and storage 13
Step 1. Create the database (RDS) 13
Step 2. Create the storage and server permission 14
Step 3. Prepare the app for RDS and S3 15
PHASE 5. The Linux Server 16
Step 1. Launch the bastion host and the private app server 16
Step 2. Harden the SERVERS 17
Step 3. Deploy the application 18
Step 4. Configure Nginx and log rotation 18
PHASE 6. Load Balancer and Public Access 19
Step 1. Target Group and Application Load Balancer 19

Step 2. Add CloudFront and HTTPS....20
PHASE 7. Backup and Recovery....21
Step 1. Configure the backup script....21
Step 2. The cron schedule....22
Step 3. Lifecycle rule for backups....23
Step 4. Verify Backup Integrity....23
Step 5. Test Database Recovery....23
PHASE 8. Monitoring and Logging....24
Step 1. CloudWatch agent....24
Step 2. Configure Security Alerts Using Amazon CloudWatch and SNS....25
Step 3. Health and Uptime Alarms....26
Step 4. Create an Amazon CloudWatch Monitoring Dashboard....27
PHASE 9. Docker and CI/CD....28
Step 1. Create and Test Dockerfiles....28
Step 2. GitHub actions CI pipeline....29
Step 3. Automated deployment....29
Step 3.1. Automated Deployment Verification:....30
PHASE 10. Finalization....30
Step 1. Application Screenshots....30
Step 2. Security summary....32
Conclusion....33

PHASE 1. Planning and Setup

Step 1. Project Plan and Architecture

Decided the project name, region, and network range, and the architecture diagram. Every AWS resource in this project depends on this plan. Deciding the layout first avoids mistakes and rework later, and the diagram shows how users, the application, and the database connect.

- **Project name:** SecureDesk – IT Support Ticket System

- **Objective:** A simple internal helpdesk tool, Employees raise tickets and admins manage them.
- **Application theme:** IT Support Ticket System
- **Tech stack:** Python/Flask, MySQL (RDS), Nginx, Gunicorn, Docker, GitHub Actions
- **AWS services used:** VPC, EC2, RDS, S3, ALB, NAT Gateway, CloudFront, CloudWatch, IAM, SNS, Systems Manager
- **Region:** ap-south-1

Project Repository

The source code and project documentation for SecureDesk are maintained in GitHub.

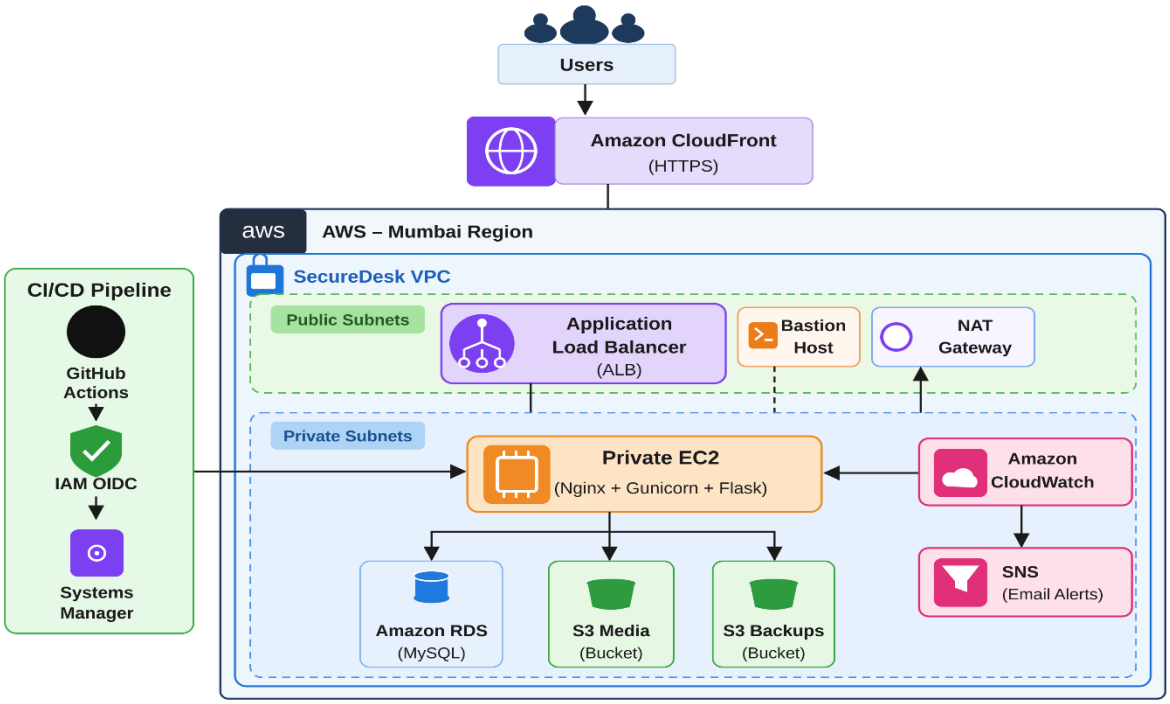
GitHub Repository:
https://github.com/dell09-cloud/securedesk-devsecops

The repository contains the application source code, Docker configuration, CI/CD workflow, deployment scripts, and AWS architecture documentation.

Live Application URL:
https://d39qz29rzn1p6r.cloudfront.net

The SecureDesk application was deployed on AWS and made accessible through Amazon CloudFront using HTTPS. AWS resources may be removed after project completion, therefore; the application URL may no longer remain accessible.

Step 2. Architecture Diagram and Explanation



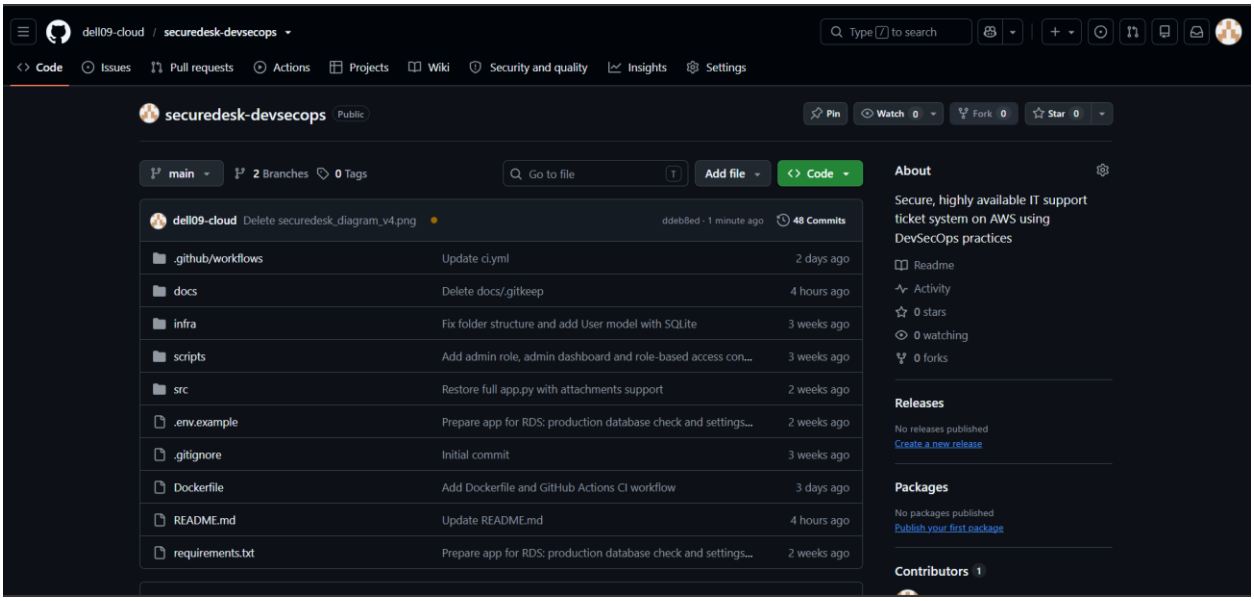
Explanation: SecureDesk architecture. Users reach the application through CloudFront and an Application Load Balancer. The Flask app runs on an EC2 instance in a private subnet, and the MySQL database runs in separate private data subnet. S3, CloudWatch, SNS and Systems Manager are regional AWS services outside the VPC, reached through the server's IAM role.

Security: The design keeps the app server and database in private subnets, with the load balancer as the only public entry point and a bastion host (my IP only) for SSH.

- **Traffic flow:** Users → CloudFront (HTTPS) → ALB → Nginx → Gunicorn → Flask Application → RDS (MySQL)
- **Why public vs private subnets:** The load balancer is in the public subnets and is the only entry point for users. The EC2 server and the database are in private subnets, so they can't be reached directly from the internet.
- **Why a NAT Gateway:** The NAT Gateway lets the private server download updates without being exposed.
- **Why a bastion host:** The bastion host is the only way to SSH into the server, and only from my IP address.
- **Role of S3 and CloudFront:** S3:- private buckets files reach users only through the application after an ownership check. CloudFront:- the public HTTPS entry point and DNS name, so no Route 53 is needed. It redirects HTTP to HTTPS, adds HSTS and works with WAF. Caching is off because pages are per-user.

Step 3. Create the GitHub repository

Creating an empty home for project code, scripts, and documentation.



GitHub repository securedesk-devsecops with the project structure: src, infra, scripts, and docs.

Security: The Python .gitignore keeps the virtual environment and local database out of the repository, since the repo is public and secrets must not be uploaded.

Step 4. Set up the local development environment

Installed Python, Git, and VS Code, cloned the repository to my laptop, and created a Python virtual environment with Flask installed.

Step 5. Create the first Flask page

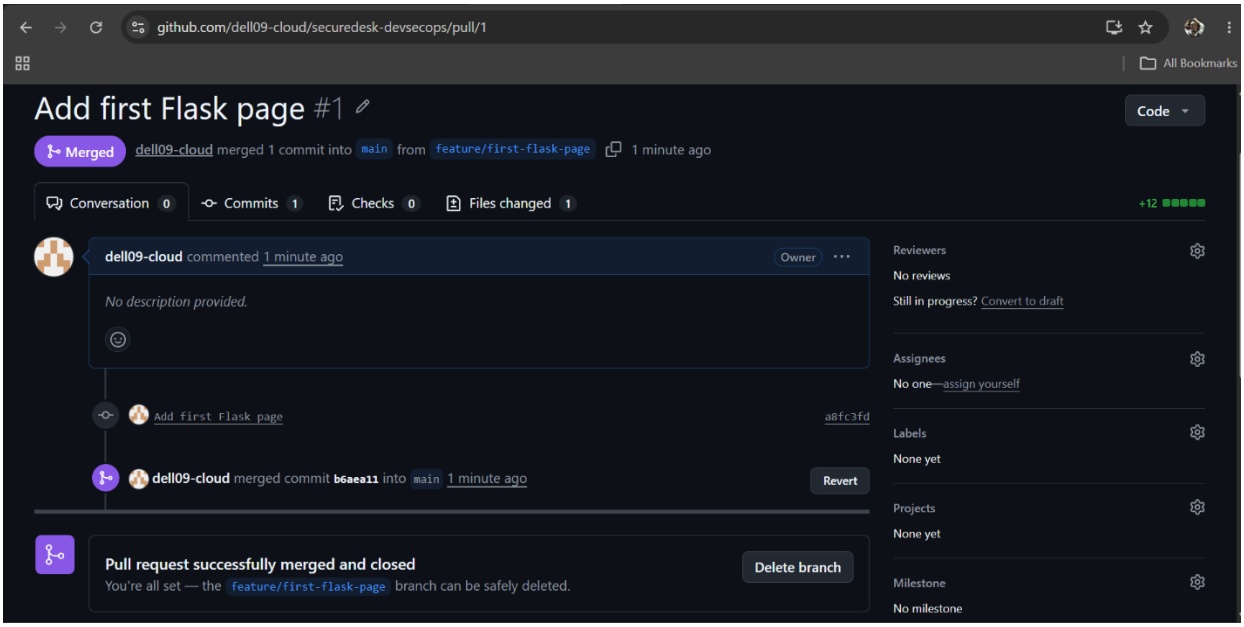
Wrote a small Flask app in *src/app.py* and ran it locally to confirm the setup works.



Security: Debug mode is used only on my laptop and will be turned off on the server, because it exposes a debugging console.

Step 6. Merge changes using a branch and pull request

Committed the new code on a separate branch, pushed it to GitHub, and merged it into main through a pull request.



Pull request merged on GitHub: the first Flask page was developed on a feature branch and merged into main.

Security: Changes are developed on a separate branch and merged into main through a pull request, so main only receives changes that went through a pull request.

PHASE 2. Application Development

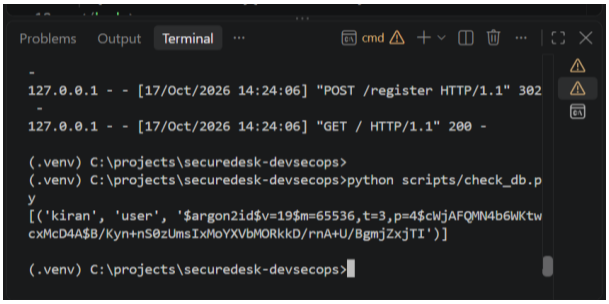
Step 1. Create the Database and User model

Created a new branch, feature/user-auth, and added Flask-SQLAlchemy to the app. Defined a users table with username, email, hashed password, role (user or admin), and creation time. The database address is read from an environment variable, so the app uses SQLite on my laptop now and can switch to MySQL on RDS during deployment without code changes.

Security: The database address and secret key are read from environment variables, so real values stay out of the code. Usernames and emails are unique at the database level, and the role column (user or admin) supports access control later.

Step 2. Build user registration with password hashing

Added a /register page with server-side validation of the username, email, and password length. Passwords are hashed with argon2 before being saved, and only the hash is stored in the database. Duplicate usernames and emails are rejected cleanly



The users table stores only the argon2 hash of each password, never the password itself.

Security: Passwords are hashed with argon2 and a random salt, so only the hash is stored. User input is checked on the server before saving, and SQLAlchemy sends it as parameters, which prevents SQL injection.

Step 3. Add login, logout, and sessions

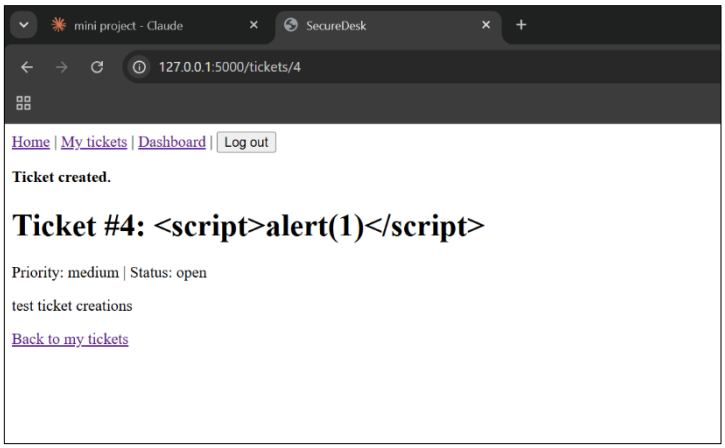
Added a /login page that checks the password against the argon2 hash, a logout button, and a dashboard page that only logged-in users can open. Login errors use one shared message, so usernames can't be guessed. Added CSRF tokens to all forms, and secured the session cookie with HttpOnly and SameSite settings.

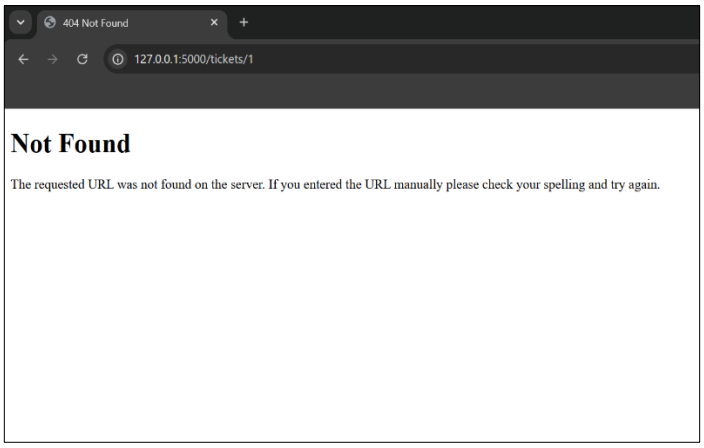


Security: Passwords are checked against the hash, and wrong logins show one shared message so usernames can't be guessed. The session cookie is HttpOnly and SameSite. All forms have CSRF tokens, logout is a POST request, and the dashboard requires login.

Step 4. Add tickets: create, list, and view

Added a tickets table linked to users, with pages to create a ticket, list your own tickets, and view one. Input is checked on the server. And also added to edit a ticket and buttons to close or delete, completing create, read, update, and delete. Close and delete accept only POST requests with a CSRF token, and every action checks that ticket belongs to the logged-in user





Security: Each user can only open their own tickets, and another user's ticket gives a "not found" page, so ticket numbers can't be guessed. Text typed into a ticket is escaped when displayed.

Step 5. Add the admin role, dashboard, and admin actions

Added an admin role with a dashboard showing the number of users, total tickets, and open and closed tickets, plus pages listing all users and all tickets. Admins can delete users and close or delete any ticket.



Security: Registration always creates normal users, and admins are created only by a script run on the server, never from the website. Every admin page checks the role on the server, so a normal user who types the address gets a 403 Forbidden page. All admin actions are POST requests with a CSRF token, and an admin can't delete their own account or another admin.

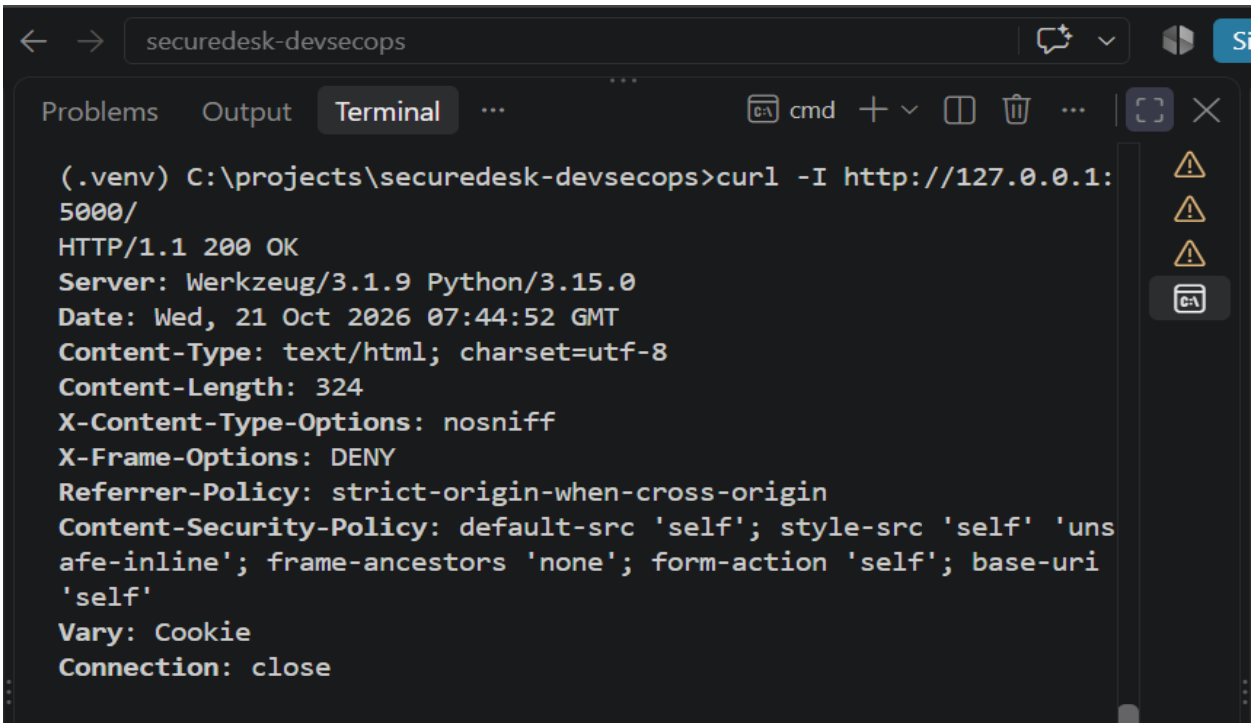
Step 6. Add rate limiting and brute-force protection

Added rate limits with Flask-Limiter: at most 10 login attempts per minute from one IP address, and at most 5 failed attempts per 15 minutes for one username. Registration is limited to 10 attempts per hour. Extra attempts get a 429 "Too many attempts" response.



Step 7. Add security headers and production settings

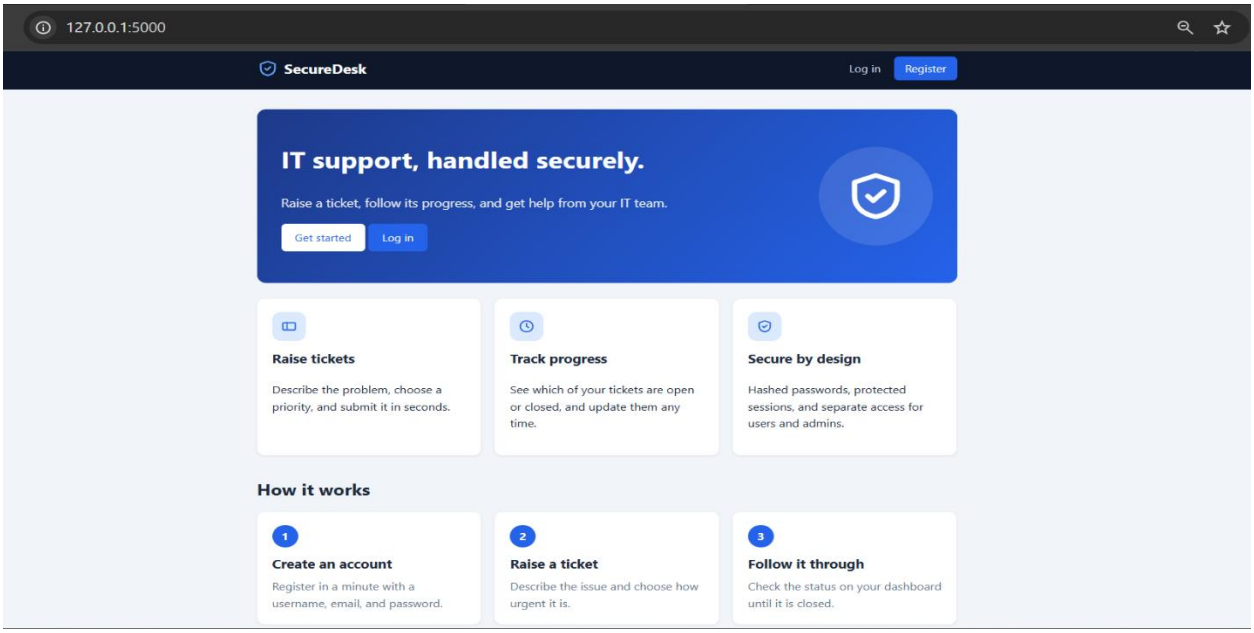
Added security headers to every response: X-Content-Type-Options, X-Frame-Options, Referrer-Policy, and a Content-Security-Policy that only allows scripts from the site itself. HSTS is added by cloudfront. Debug mode and the secret key are controlled by environment settings, and in production the app refuses to start without a real secret key and turns on the Secure cookie flag.



The application sends security headers on every response, including X-Frame-Options and a Content-Security-Policy.

Step 8. Design the user interface

Styled the app with one local stylesheet and inline SVG icons, with no external libraries, so it meets the Content-Security-Policy. Added a home page, dashboards with statistic cards, colored status badges, and friendly error pages.



Home page of SecureDesk, showing the redesigned interface with the hero banner, feature cards, and a "How it works" section.

PHASE 3. AWS Networking

Step 1. Account Safety

Turned on multi-factor authentication for the root user, created a separate IAM user with MFA for daily work, and set up a monthly budget alert so unexpected costs are noticed early. All resources are created in one region (ap-south-1) mumbai.

Step 2. Build the network (VPC, subnets, and routing)

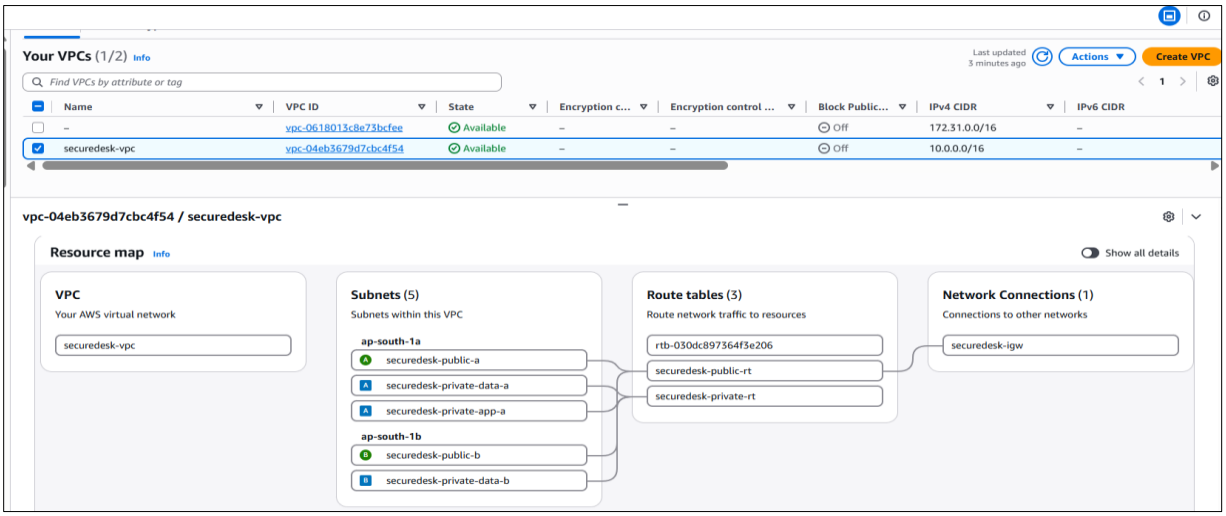
Created a custom VPC named securedesk-vpc with the address range 10.0.0.0/16, with DNS hostnames enabled, so the project has its own isolated network. Divided it into five subnets across two Availability Zones, so the load balancer and database can work in two zones.

Subnet	Availability Zones	CIDR	Purpose
securedesk-public-a	ap-south-1a	10.0.1.0/24	NAT Gateway, bastion host, load balancer
securedesk-public-b	ap-south-1b	10.0.2.0/24	Load balancer (second zone)

securedesk-app-a	ap-south-1a	10.0.11.0/24	Application server
securedesk-data-a	ap-south-1a	10.0.21.0/24	Database
securedesk-data-b	ap-south-1b	10.0.22.0/24	Database (second zone)

Attached an Internet Gateway to the VPC. A public route table sends internet traffic (0.0.0.0/0) through the gateway and is attached to the two public subnets.

The application subnet uses a private route table with a default route through the NAT Gateway. The database subnets remain private and do not have direct internet access.



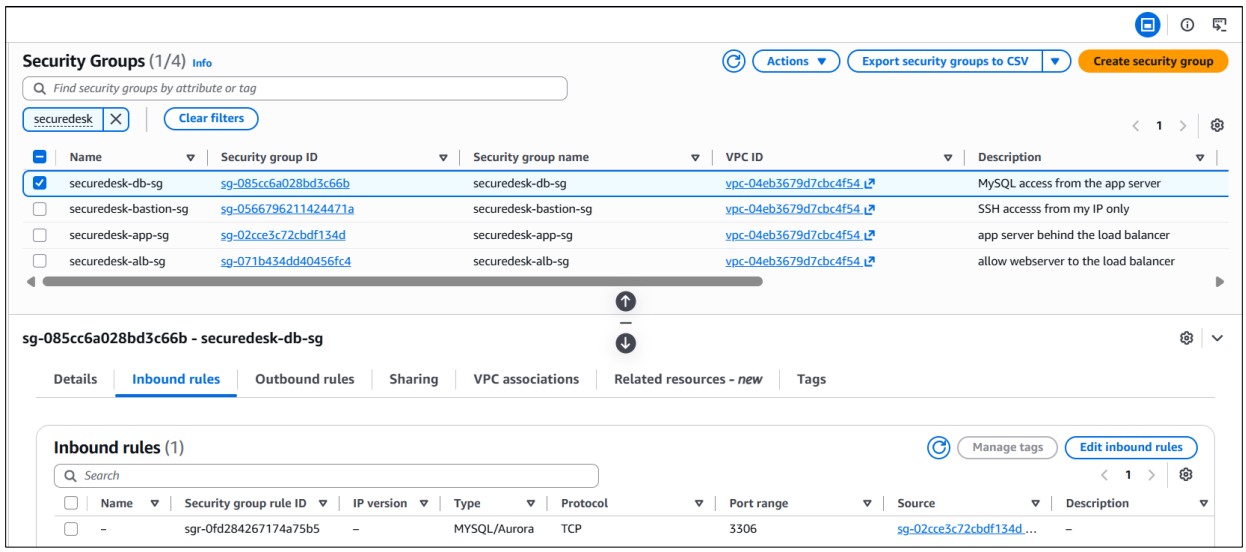
Security: Only the public subnets can be reached from the internet, so the application server and database stay hidden.

Step 3. Creating the security group

Created four security groups following the principle of least privilege, so each resource only accepts traffic from the one that needs to reach it.

Security groups	Allowed inbound	Source
securedesk-alb-sg (load balancer)	HTTP 80	Cloudfront address list
securedesk-bastion-sg (jump host)	SSH 22	My IP
securedesk-app-sg (app server)	HTTP 80	Load balancer groups
	SSH 22	Bastion group
securedesk-db-sg (database)	MySQL 3306	App server group

Security: The database accepts connections only from the application server, and the application server only from the load balancer. SSH is reachable only through the bastion host, and only from my own IP address.



PHASE 4. Database and storage

Step 1. Create the database (RDS)

Created a managed MySQL database named securedesk-db with Amazon RDS. It uses a DB subnet group that contains only the two private data subnets, one in each Availability Zone, so the database is never placed in a public subnet.

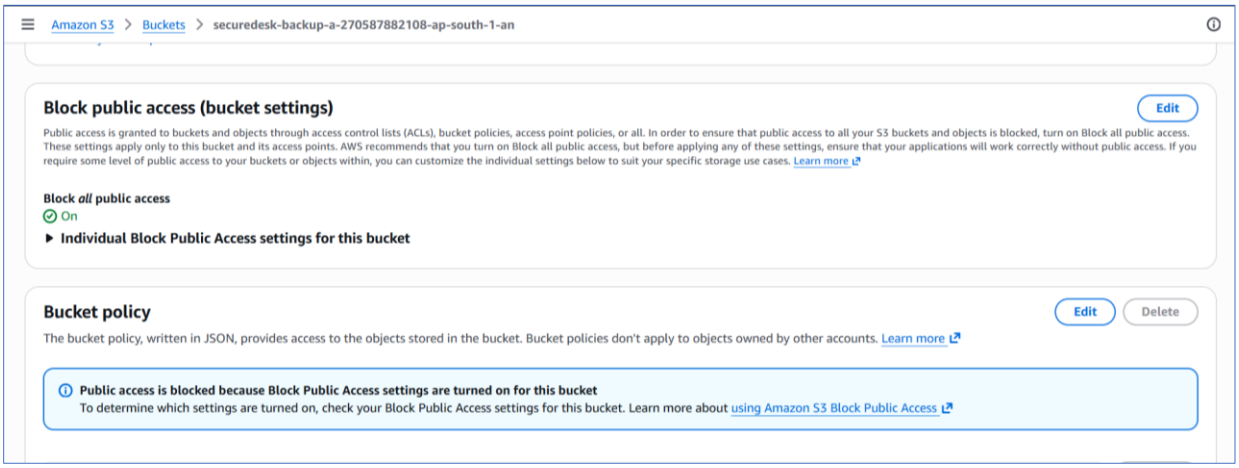


Security: The database has no public address, and its security group accepts MySQL connections only from the application server's security group, so nothing on the internet can reach it. Data is encrypted at rest. The application does not use the master account. It connects

with its own limited user, and the master password is kept outside the repository and used only for administration.

Step 2. Create the storage and server permission

Created two private S3 buckets in ap-south-1. One stores ticket attachments uploaded by users, and the other stores database backups. They're separate so that each can have its own access rules: the backups bucket stays completely private, and uploads are delivered only through the application after an ownership check.



The backup bucket blocks all public access.

Security: Block Public Access is turned on for both buckets, and both are encrypted at rest with S3-managed keys. Versioning on the backups bucket means an older copy can be recovered if a backup is overwritten or deleted by mistake.

The application server gets access to the buckets through an IAM role, so there are no long-lived secrets in the code or on the server. The role allows S3 access to these two buckets only (the backups bucket allows no deletes), plus the CloudWatch agent permissions.

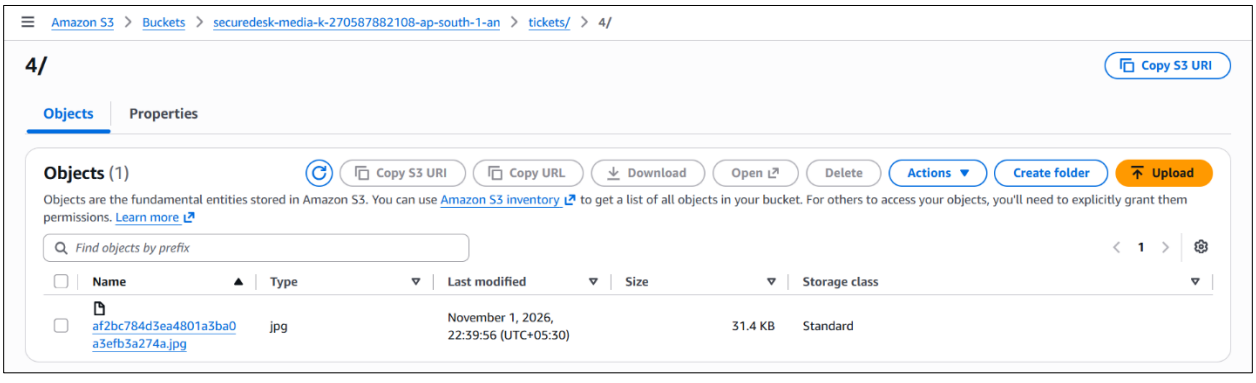
IAM > Policies > securedesk-s3-access

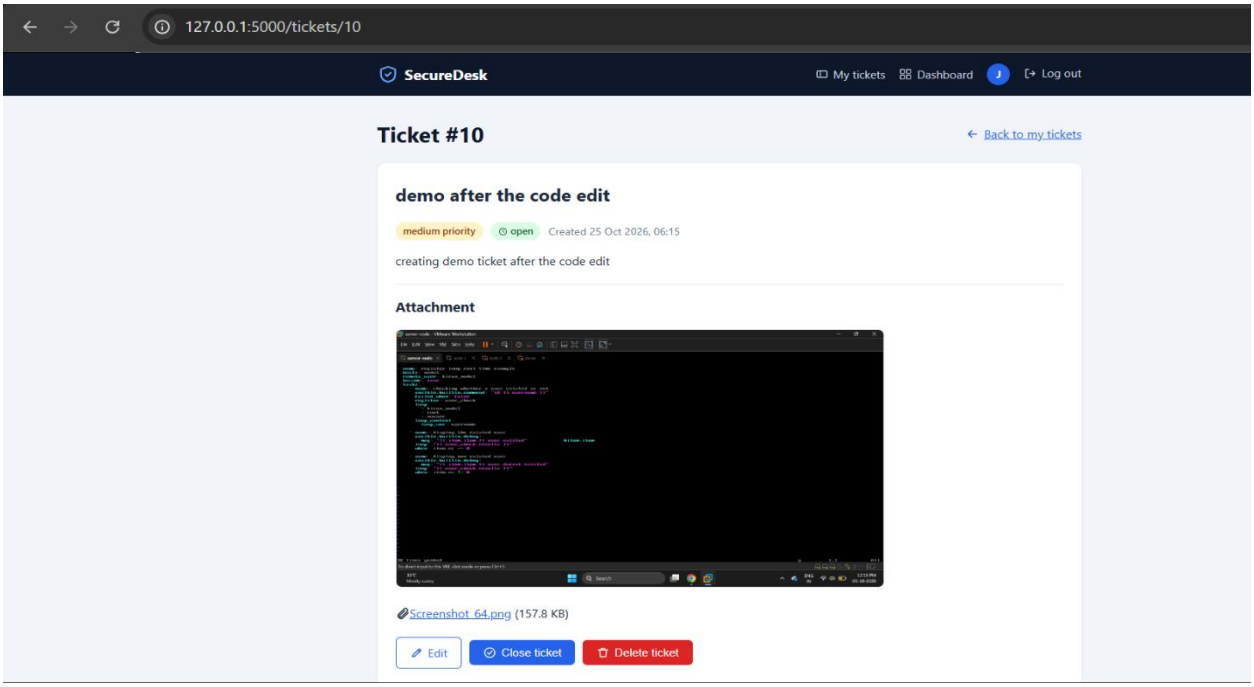
```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "MediaBucketObjects",
            "Effect": "Allow",
            "Action": [
                "s3:GetObject",
                "s3:PutObject",
                "s3:DeleteObject"
            ],
            "Resource": "arn:aws:s3:::securedesk-media-k-270587882108-ap-south-1-an/*"
        },
        {
            "Sid": "MediaBucketList",
            "Effect": "Allow",
            "Action": "s3:ListBucket",
            "Resource": "arn:aws:s3:::securedesk-media-k-270587882108-ap-south-1-an"
        },
        {
            "Sid": "BackupBucketObjects",
            "Effect": "Allow",
            "Action": [
                "s3:GetObject",
                "s3:PutObject"
            ],
            "Resource": "arn:aws:s3:::securedesk-backup-a-270587882108-ap-south-1-an/*"
        },
        {
            "Sid": "BackupBucketList",
            "Effect": "Allow",
            "Action": "s3:ListBucket",
            "Resource": "arn:aws:s3:::securedesk-backup-a-270587882108-ap-south-1-an"
```

IAM policy for the application server. It allows access only to the two project buckets, and the backups bucket has no delete permission.

Step 3. Prepare the app for RDS and S3

Made the application ready to run on AWS without changing how it works. The database address is now a setting, so the same code uses SQLite on my laptop and MySQL on RDS in production. Added a MySQL driver and connection health checks, because RDS can close idle connections. Added ticket attachments: a file chosen on the new ticket form is stored in the S3 media bucket, and a new attachments table records which file belongs to which ticket. During development, files are saved to a local folder instead, because the server's IAM role only exists on the server.





A ticket with an attached image, shown in the application. The upload is checked on the server before it is stored.

Security: The application reaches S3 through the server's IAM role, so there are no access keys in the code. In production, the app refuses to start unless the database address, secret key, and bucket name are set as environment settings. A template file, .env.example, lists the required settings with placeholder values only, and real values are kept out of GitHub.

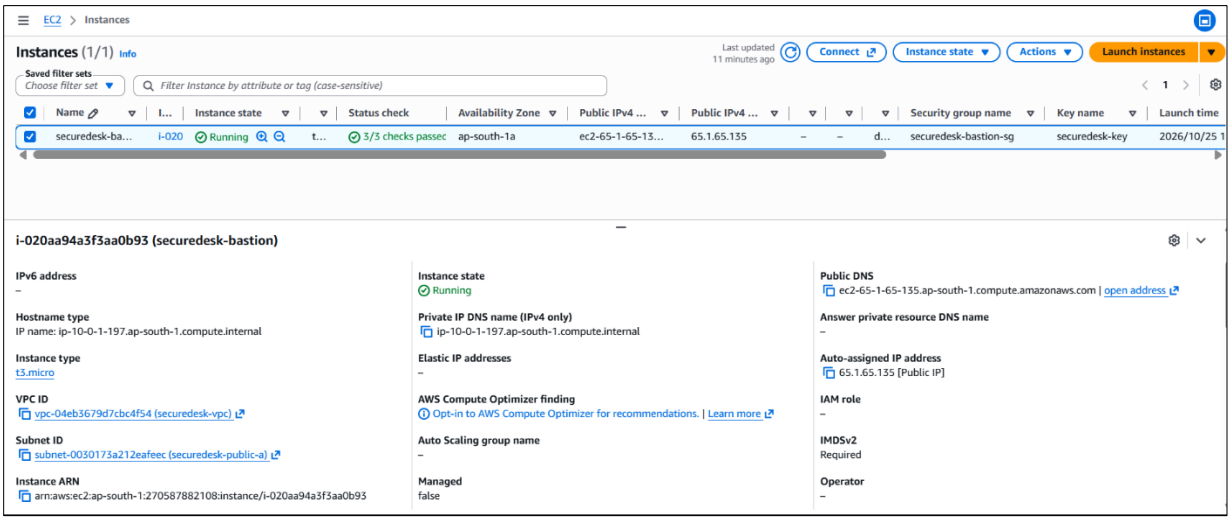
PHASE 5. The Linux Server

Step 1. Launch the bastion host and the private app server

Launched a small Ubuntu 24.04 server as a bastion host in the public subnet. It accepts SSH only from my IP address and is the only way into the private network. Launched the application server in a private subnet with no public IP address. It has the IAM role for S3 access, and the instance metadata service is set to the more secure version 2.

A NAT Gateway in the public subnet lets the private server download updates without being reachable from the internet. Logins use an SSH key pair, and the private server is reached through the bastion.





Step 2. Harden the SERVERS

Hardened SSH on both servers: root login and password login are disabled, only the ubuntu account may log in, login attempts and idle time are limited, and only key-based authentication works. Installed fail2ban to block addresses that repeatedly fail to log in. Enabled the UFW firewall as a second layer behind the security groups: the app server allows SSH only from the bastion subnet and web traffic only from the load balancer subnets, and the bastion allows only SSH. Changes were tested from a second session before the first was closed, to avoid being locked out.

```
ubuntu@ip-10-0-11-171:~$ sudo sshd -T | grep -E "^(permitrootlogin|passwordauthentication|maxauthtries|allowusers)"
maxauthtries 4
permitrootlogin no
passwordauthentication no
allowusers ubuntu
ubuntu@ip-10-0-11-171:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

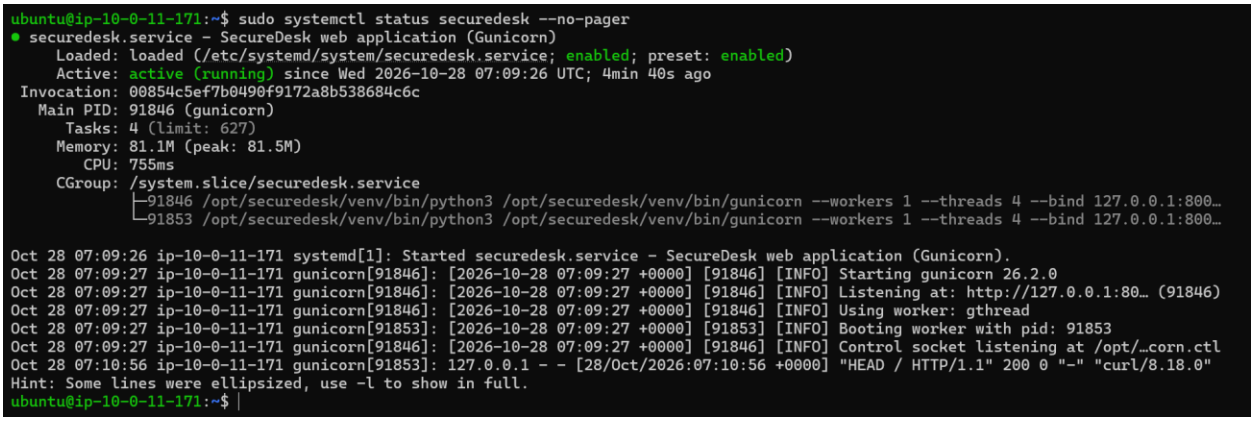
To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    10.0.1.0/24               # SSH from bastion subnet
80/tcp                     ALLOW IN    10.0.1.0/24               # HTTP from load balancer
80/tcp                     ALLOW IN    10.0.2.0/24               # HTTP from load balancer

ubuntu@ip-10-0-11-171:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     0
|  `- Journal matches:  _SYSTEMD_UNIT=ssh.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     0
   `- Banned IP list:
ubuntu@ip-10-0-11-171:~$
```

The app server's hardened settings: root and password logins are disabled, the firewall allows only the bastion and load balancer subnets, and fail2ban is protecting SSH.

Step 3. Deploy the application

Installed the application on the private server from the GitHub repository, into its own folder with its own Python environment, and installed Gunicorn, a production web server. The app runs under a dedicated system account that has no password and no login shell, so a compromise of the app would not give access to the main account. Created a limited database user on RDS that can only read and write the securedesk database, can only connect from the app server's subnet, and must use an encrypted connection that verifies the RDS certificate. The application's settings (secret key, database address, bucket names) are stored in a file readable only by the system's root account, outside the code and outside GitHub. systemd runs the app as a service that starts at boot and restarts automatically if it crashes, with extra restrictions on what the process may do. Gunicorn listens only on the server's own address, so the web server in front will be the only way in. On its first start, the app created its tables in the RDS database. Nginx is the only way into the application on the server



Security: least-privilege database user restricted by subnet, encrypted and verified database connection, secrets kept in a root-only file and never in the repository, and a service account with no login.

Step 4. Configure Nginx and log rotation

Put Nginx in front of the application as a reverse proxy, so it is the only service accepting web traffic on the server. Gunicorn listens only on the server's own loopback address. Nginx hides its version number, rejects uploads over 6 MB before they reach the app, passes the real visitor address and protocol to the app, and exposes a lightweight health check address for the load balancer. Log rotation is in place for every log source: Nginx logs rotate daily, are compressed, and keep 14 days of history, and the application logs in the system journal are limited to 200 MB and 14 days. Rotation was tested with a forced run.

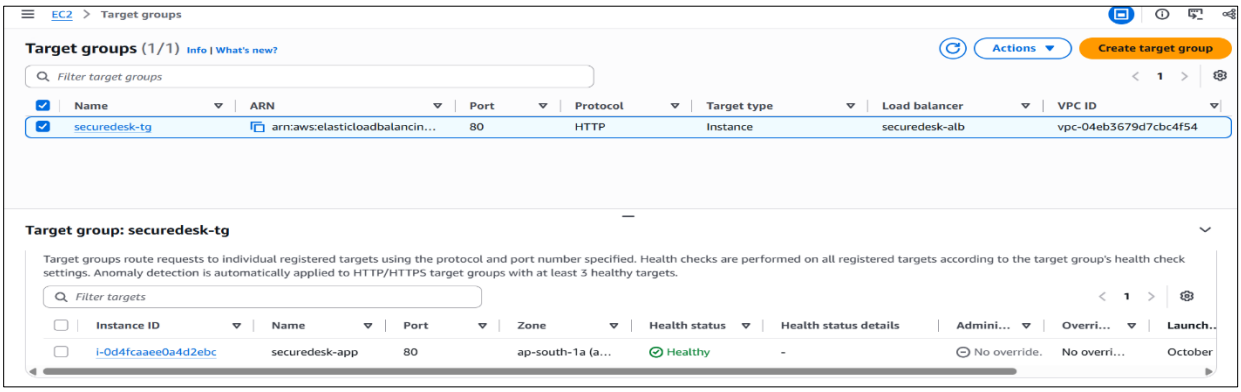
```
ubuntu@ip-10-0-11-171:~$ ls -l /var/log/nginx/
total 12
-rw-r----- 1 www-data adm  87 Oct 28 07:39 access.log
-rw-r----- 1 www-data adm 341 Oct 28 07:33 access.log.1
-rw-r----- 1 www-data adm   0 Oct 28 07:39 error.log
-rw-r----- 1 www-data adm 258 Oct 28 07:32 error.log.1
ubuntu@ip-10-0-11-171:~$
cat /etc/systemd/journald.conf.d/securedesk.conf
[Journal]
SystemMaxUse=200M
MaxRetentionSec=14day
Compress=yes
ubuntu@ip-10-0-11-171:~$
journalctl --disk-usage
Archived and active journals take up 143.1M in the file system.
ubuntu@ip-10-0-11-171:~$
ubuntu@ip-10-0-11-171:~$
```

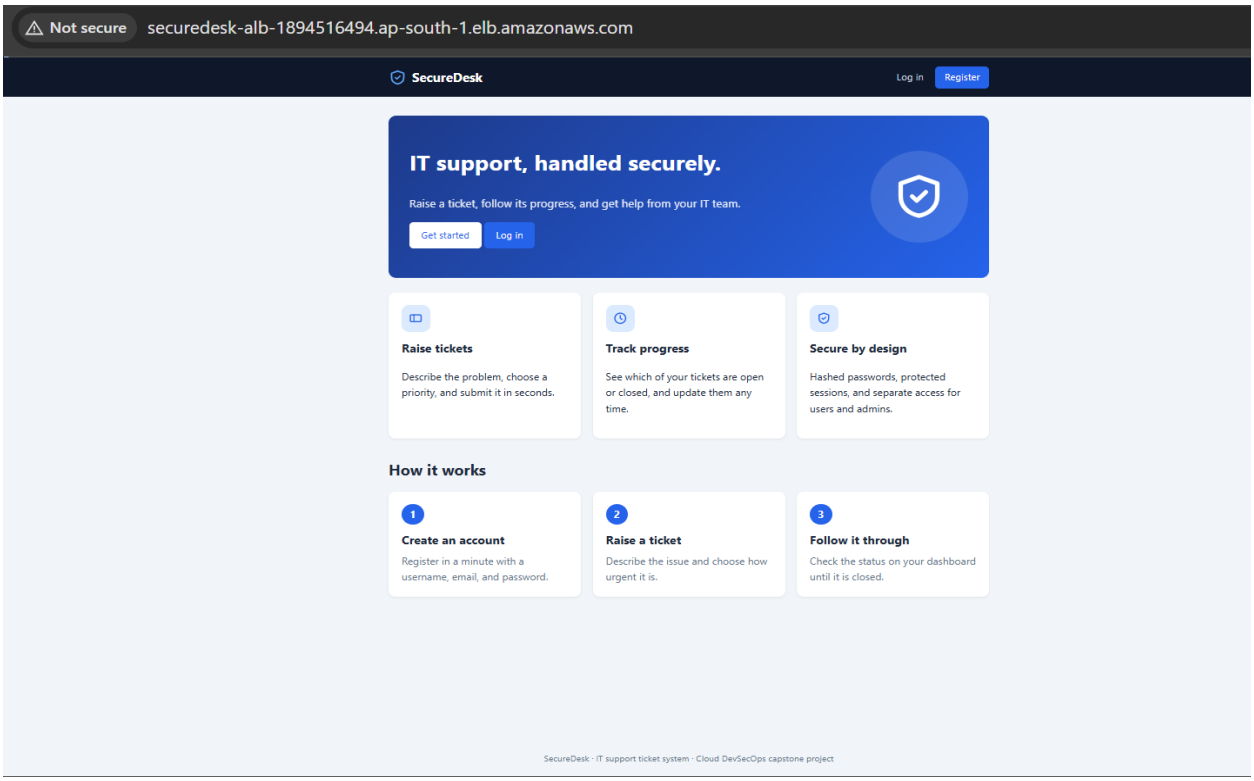
PHASE 6. Load Balancer and Public Access

Step 1. Target Group and Application Load Balancer

Created an internet-facing Application Load Balancer in the two public subnets, one in each Availability Zone. It's the only public entry point to the application. It forwards traffic to a target group containing the application server in the private subnet. The target group checks the server every 15 seconds on a dedicated /healthz page, which Nginx answers on its own, so the check doesn't load the application or the database.

Component	setting
Load balancer	securedesk-alb, internet-facing, public subnets in both zones
Security group	securedesk-alb-sg, HTTP 80
Listener	HTTP port 80, forwards to the target group
Target group	securedesk-tg, instance target, port 80
Health check	/healthz, expects 200, every 15 seconds, healthy after 2 passes





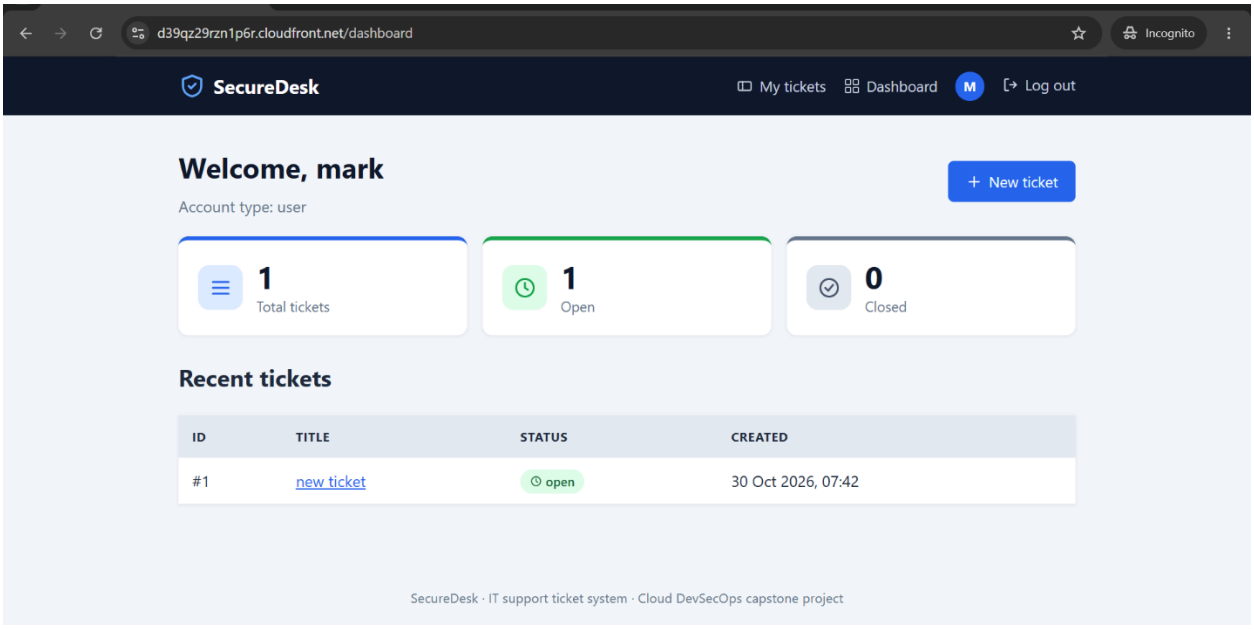
Before CloudFront was added, the ALB answered directly over HTTP.

Security: The application server has no public address. Its security group and its firewall accept web traffic only from the load balancer's group and subnets, so nobody can reach the server directly. The session cookie is marked Secure in production, so login only works over HTTPS. HTTPS is provided by CloudFront

Step 2. Add CloudFront and HTTPS

Put an Amazon CloudFront distribution in front of the load balancer. Visitors connect over HTTPS using CloudFront's own certificate, and plain HTTP requests are redirected to HTTPS. Caching is switched off, because the pages are different for every user, and all cookies, headers, and form data are forwarded to the application. The application is configured to trust two proxies (CloudFront and load balancer), so the login rate limits and the logs see the visitor's real IP address. The connection from CloudFront to the load balancer stays inside AWS's network and uses HTTP, because a certificate can't be issued for the load balancer's AWS address without a custom domain. The load balancer's security group accepts web traffic only from CloudFront's own address list, so the load balancer's direct address no longer works. CloudFront adds an HSTS header. The web firewall rule that limits request bodies to 8 KB is set to count instead of block, so file attachments can be uploaded, while the other firewall rules stay on and the application still enforces its own 5 MB limit and type checks.

```
C:\Users\dell9>
C:\Users\dell9>curl -I https://d39qz29rzn1p6r.cloudfront.net
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 3519
Connection: keep-alive
Date: Mon, 02 Nov 2026 05:22:07 GMT
Vary: Cookie
Server: nginx
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Referrer-Policy: strict-origin-when-cross-origin
Content-Security-Policy: default-src 'self'; style-src 'self' 'unsafe-inline'; frame-ancestors 'none'; form-action 'self'; base-uri '
self'
X-Cache: Miss from cloudfront
Via: 1.1 dd6c7cbe6418ff0051ecdabc288fbca4.cloudfront.net (CloudFront)
X-Amz-Cf-Pop: MAA51-P1
X-Amz-Cf-Id: _Xf90Qs71dwfOEpZZN88wWEcSOorf7yqPtAHr4sCUa0au1LzbOIXVA==
Strict-Transport-Security: max-age=31536000
```



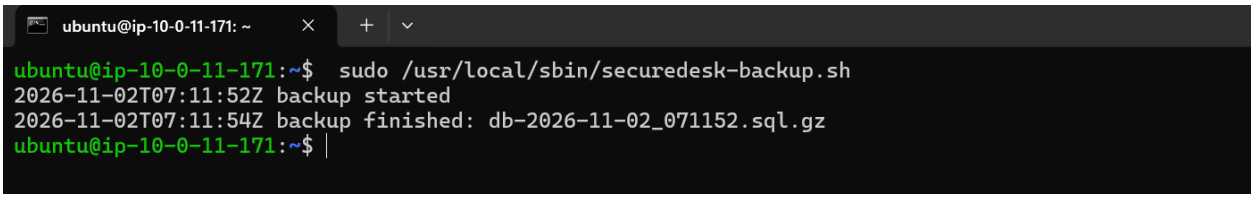
PHASE 7. Backup and Recovery

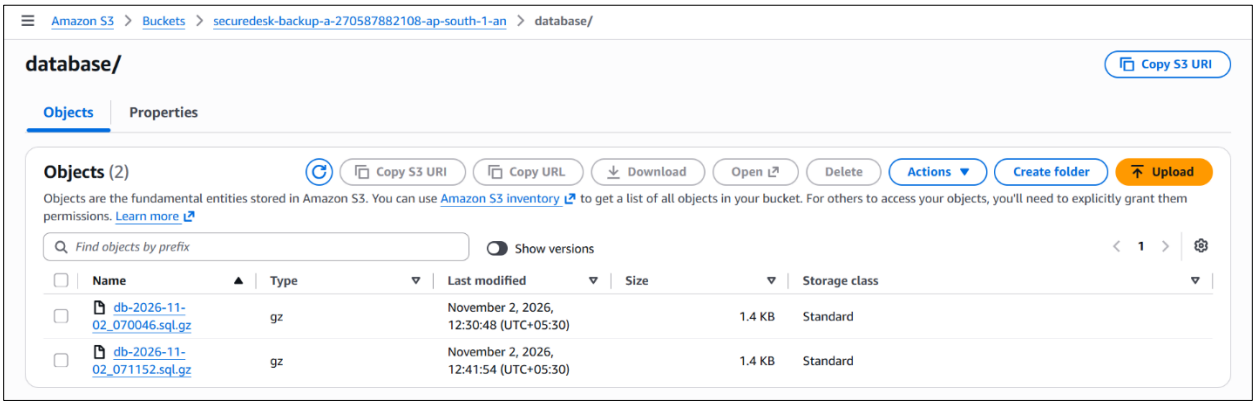
Step 1. Configure the backup script

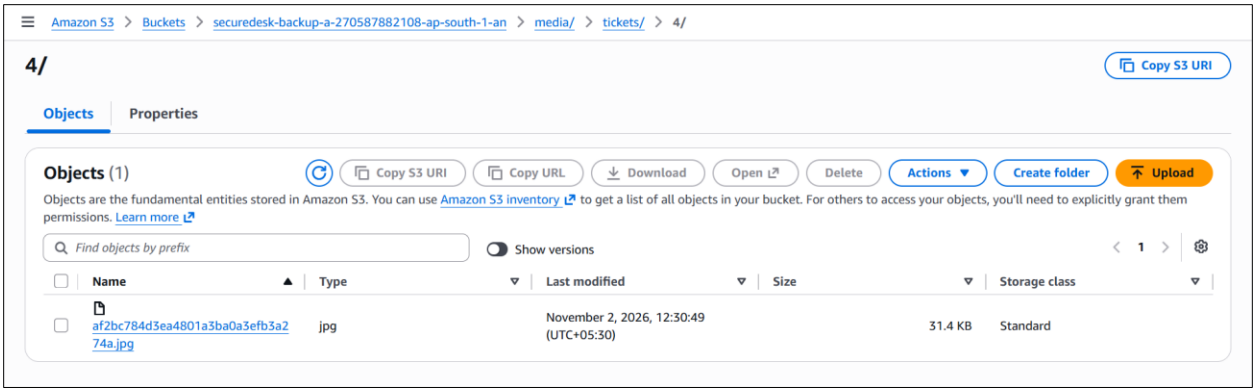
Created a backup script named securedesk-backup.sh on the private application server to protect the SecureDesk database and uploaded files. The script uses mysqldump to create compressed MySQL database backups and uploads them to the private Amazon S3 backups bucket. It also synchronizes uploaded media files from the media bucket to the backups bucket. The backup process uses a dedicated MySQL backup account with restricted permissions. Database credentials are stored in a protected configuration file outside the application repository, and the RDS connection uses SSL certificate verification.

Verified the successful upload of database backup files and ticket attachment files to the private Amazon S3 backups bucket. The database backups were stored in compressed .sql.gz format, while uploaded media files were synchronized into the media backup

directory. The S3 listing confirmed the presence of two database backup files and one media attachment file.

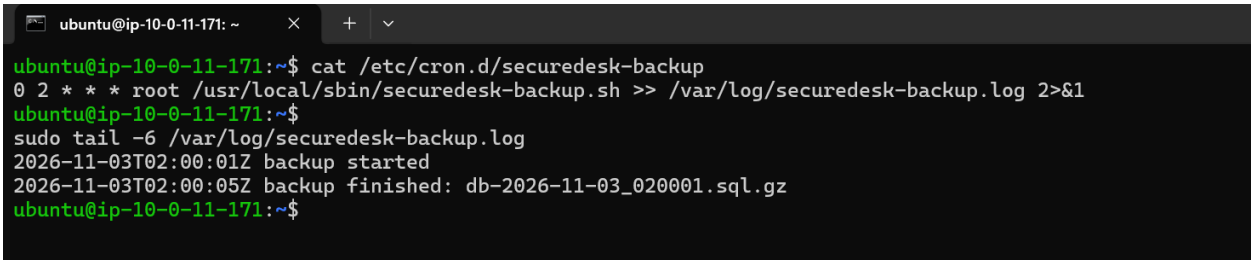






Step 2. The cron schedule

A cron job runs the script daily at 02:00 UTC and writes to a log that rotates weekly. I tested the schedule by temporarily running it every two minutes and checking the log, then restored the daily schedule.

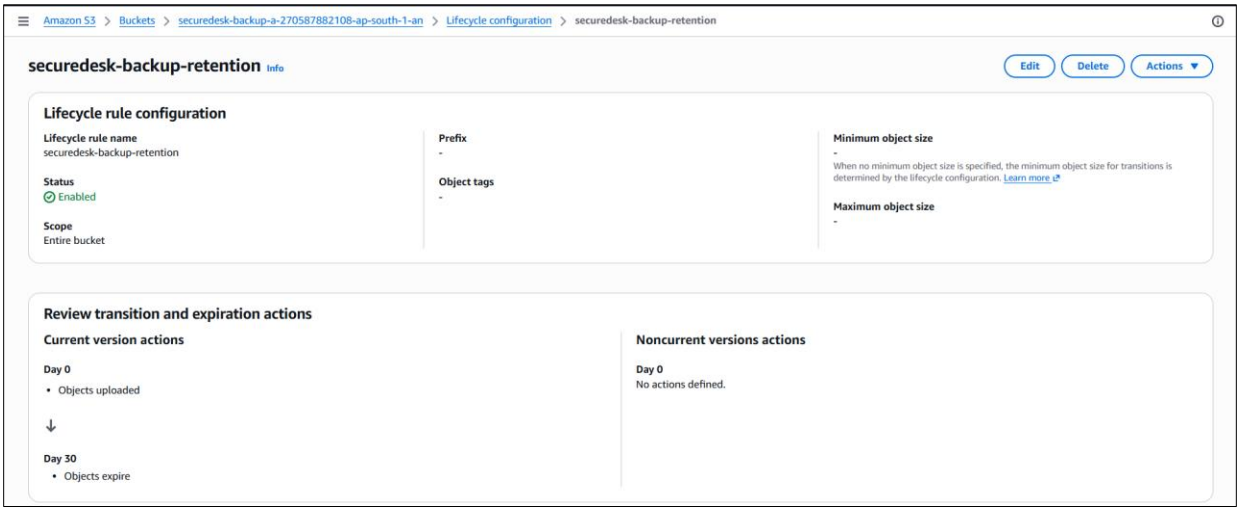


```
ubuntu@ip-10-0-11-171:~$ aws s3 ls s3://securedesk-backup-a-270587882108-ap-south-1-an/media/ --recursive --region ap-south-1
2026-11-02 07:00:49      32166 media/tickets/4/af2bc784d3ea4801a3ba0a3efb3a274a.jpg
ubuntu@ip-10-0-11-171:~$ aws s3 ls s3://securedesk-backup-a-270587882108-ap-south-1-an/database/ --region ap-south-1
2026-11-02 07:00:48       1471 db-2026-11-02_070046.sql.gz
2026-11-02 07:11:54       1471 db-2026-11-02_071152.sql.gz
ubuntu@ip-10-0-11-171:~$
ubuntu@ip-10-0-11-171:~$ aws s3 ls s3://securedesk-backup-a-270587882108-ap-south-1-an/database/ --recursive --human-readable --summa
rize --region ap-south-1
2026-11-02 07:00:48    1.4 KiB database/db-2026-11-02_070046.sql.gz
2026-11-02 07:11:54    1.4 KiB database/db-2026-11-02_071152.sql.gz

Total Objects: 2
   Total Size: 2.9 KiB
ubuntu@ip-10-0-11-171:~$
```

Step 3. Lifecycle rule for backups

Configured an Amazon S3 Lifecycle rule named securedesk-backup-retention on the private backups bucket. The rule automatically expires current backup objects after 30 days, helping manage storage usage and maintain a defined backup retention period. The lifecycle rule is enabled, and S3 versioning is retained to support recovery of previous object versions.



Step 4. Verify Backup Integrity

Downloaded a database backup from the private Amazon S3 backups bucket to the application server and performed a gzip integrity check using gzip -t. The command completed successfully and confirmed that the compressed backup archive was not corrupted.

```
ubuntu@ip-10-0-11-171:~$ gzip -t /tmp/securedesk-test.sql.gz && echo "Backup integrity check passed"
Backup integrity check passed
ubuntu@ip-10-0-11-171:~$
```

Step 5. Test Database Recovery

Performed a database recovery test using the downloaded backup file. Restored the backup to a temporary Amazon RDS database named securedesk_restore_test_20261102. Verified that the users, tickets, and attachments tables were successfully restored. After confirming the recovery, the temporary database was deleted.

Security: Database credentials are not hardcoded into the backup script or stored in GitHub. The backup files are stored in the private S3 bucket, keeping them separate from the live application data**.**

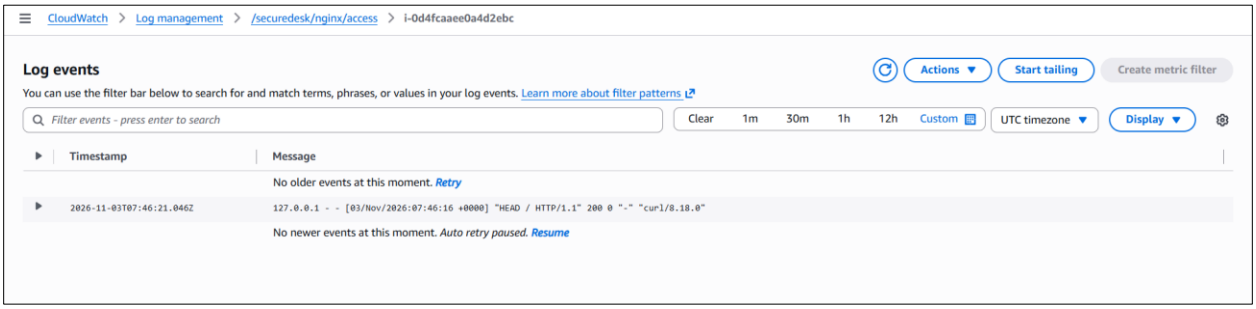
PHASE 8. Monitoring and Logging

Step 1. CloudWatch agent

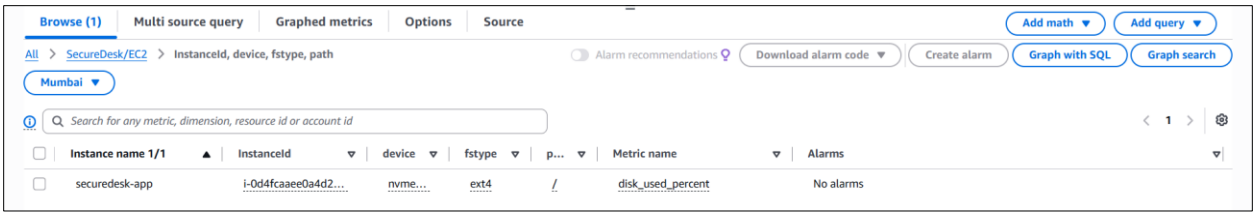
Installed and configured the Amazon CloudWatch Agent on the private SecureDesk application EC2 instance. The agent collects application logs from the securedesk.service systemd journal and Nginx access and error log files. Application and Nginx access logs were successfully delivered to Amazon CloudWatch Logs.

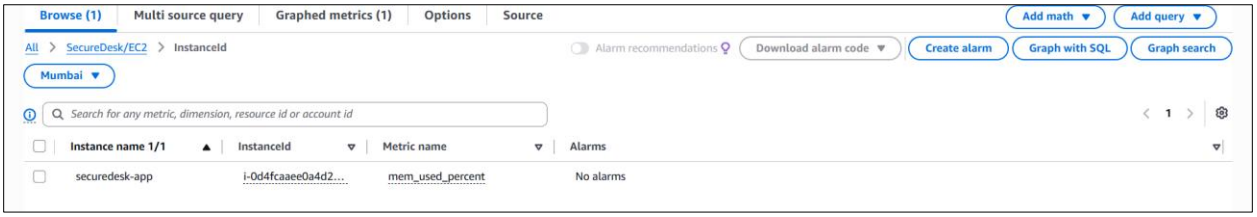






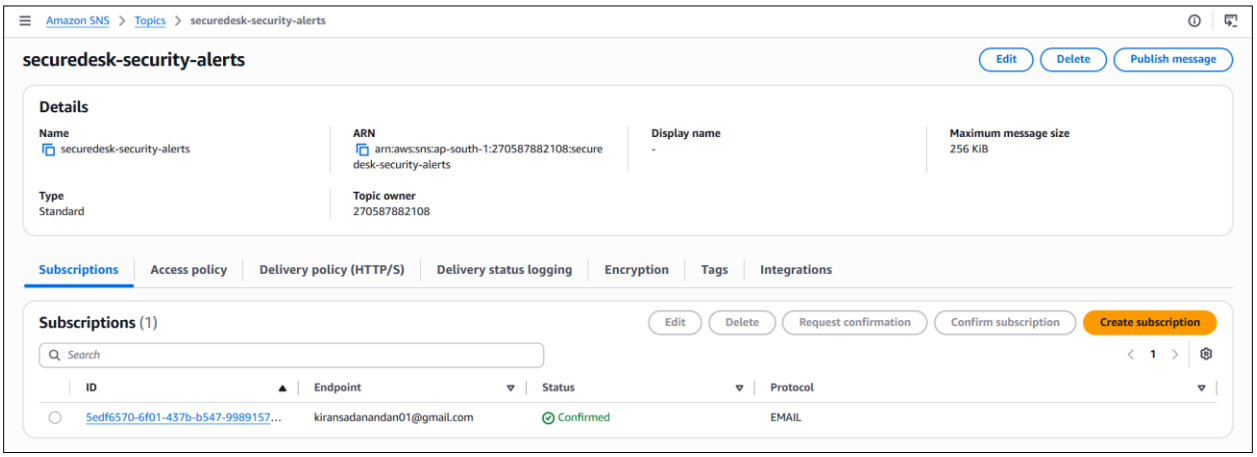
Configured system monitoring to collect memory usage (mem_used_percent) and root disk usage (used_percent) under the custom CloudWatch namespace SecureDesk/EC2, at 60-second intervals. The EC2 instance IAM role was assigned the CloudWatchAgentServerPolicy to allow the agent to publish logs and metrics to CloudWatch while retaining its existing S3 permissions. Verified the application and Nginx access log streams and confirmed that memory and disk metrics are available in CloudWatch.



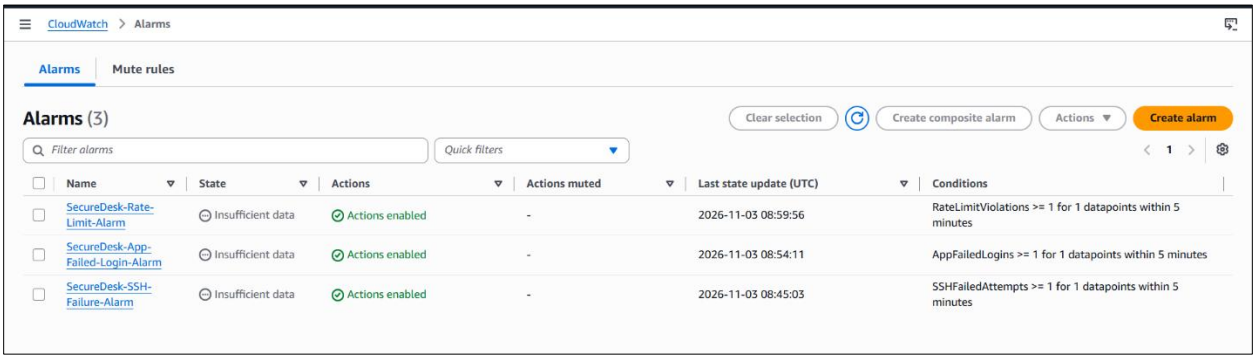


Step 2. Configure Security Alerts Using Amazon CloudWatch and SNS

Created an Amazon SNS topic named securedesk-security-alerts and confirmed the email subscription. Configured CloudWatch metric filters to detect failed SSH authentication attempts, unsuccessful application login attempts, and HTTP 429 rate-limit responses.

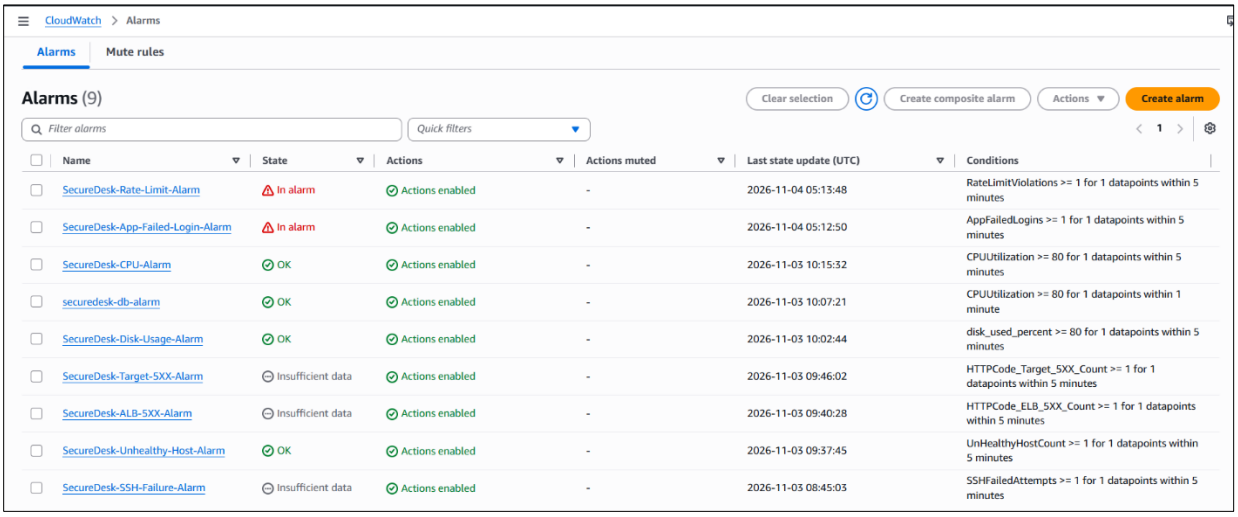


Created CloudWatch alarms for each security metric and connected them to the SNS topic. Email notifications are configured to be sent when the corresponding alarm enters the ALARM state.

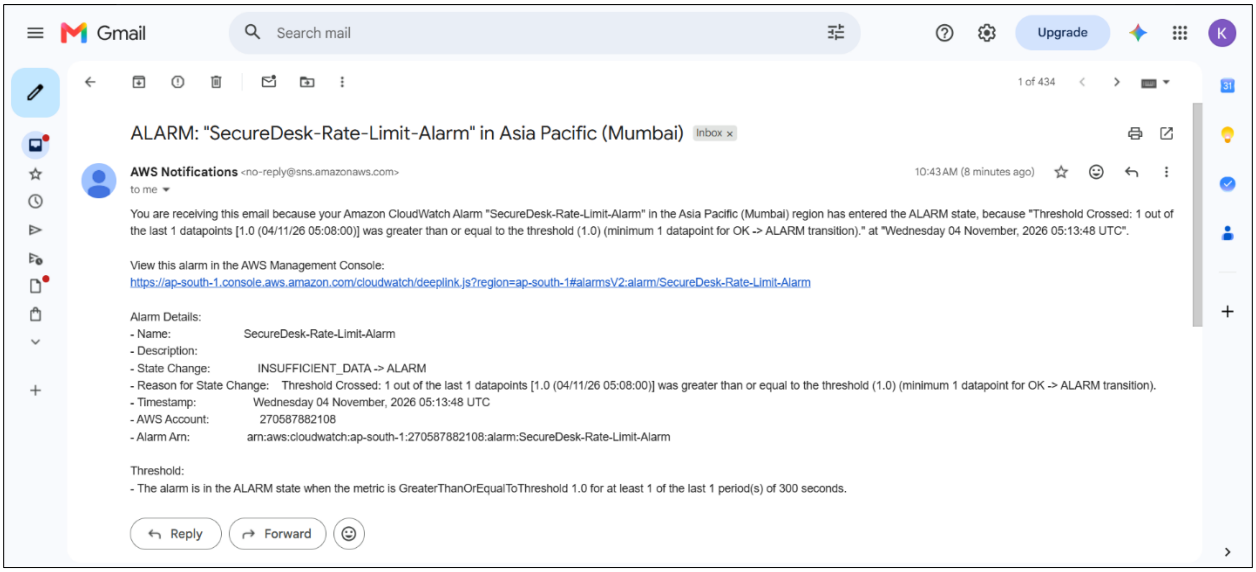


These alerts help monitor suspicious access attempts and potential abuse of the SecureDesk application.

I tested the failed-login alert by entering wrong passwords on the live site. The alarm changed to the IN ALARM state, and the email notification arrived.



The failed-login alarm changing to the ALARM state after wrong passwords were entered.



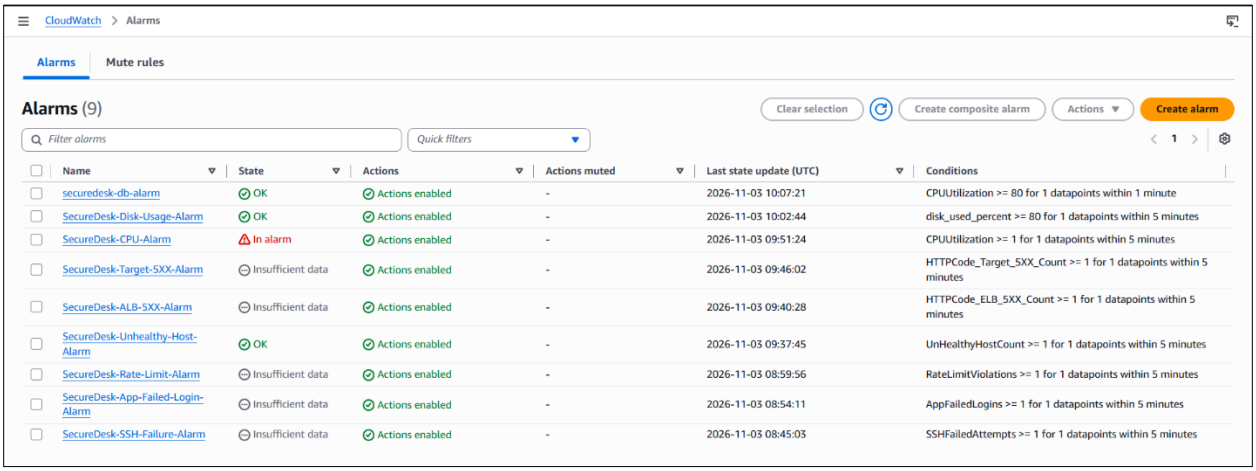
The alert email received from Amazon SNS.

Step 3. Health and Uptime Alarms

Configured Amazon CloudWatch alarms to monitor the health and performance of the SecureDesk infrastructure.

Created CloudWatch alarms to detect unhealthy targets behind the Application Load Balancer, HTTP 5xx errors generated by the load balancer and application targets, high EC2 CPU utilization, and high Amazon RDS CPU utilization.

Configured the alarms to send email notifications through the SNS topic `securedesk-security-alerts` when their respective thresholds are breached.



These alarms help identify application availability issues, server resource pressure, and database performance problems.

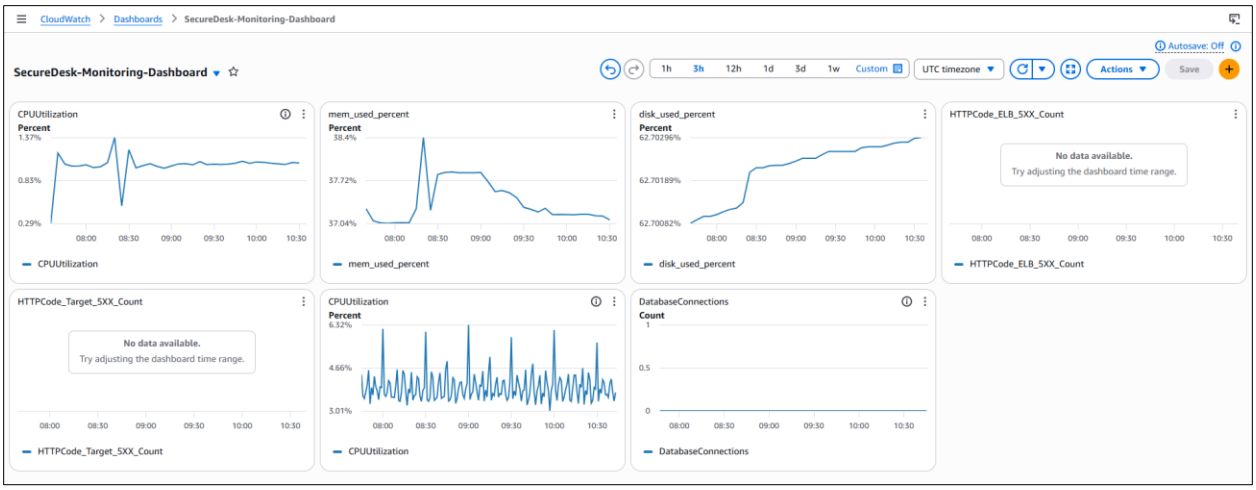
Step 4. Create an Amazon CloudWatch Monitoring Dashboard

Created an Amazon CloudWatch dashboard named SecureDesk-Monitoring-Dashboard to provide a centralized view of the SecureDesk application's infrastructure health and performance.

Added monitoring graphs for:

- EC2 CPU utilization
- EC2 memory usage
- EC2 disk usage
- Application Load Balancer 5xx errors
- Application target 5xx errors
- RDS database CPU utilization
- RDS database connections

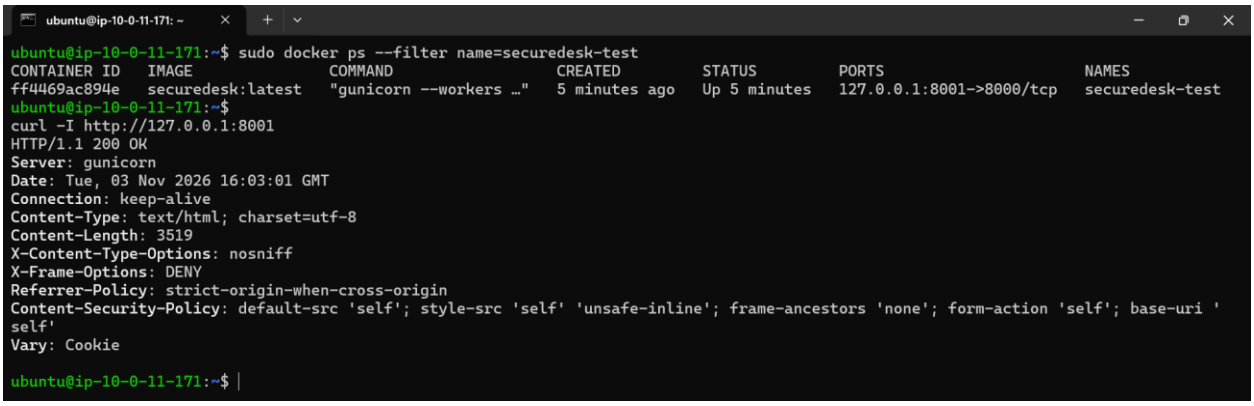
The dashboard helps monitor server resource utilization, application availability, HTTP errors, and database performance from a single interface.



PHASE 9. Docker and CI/CD

Step 1. Create and Test Dockerfiles

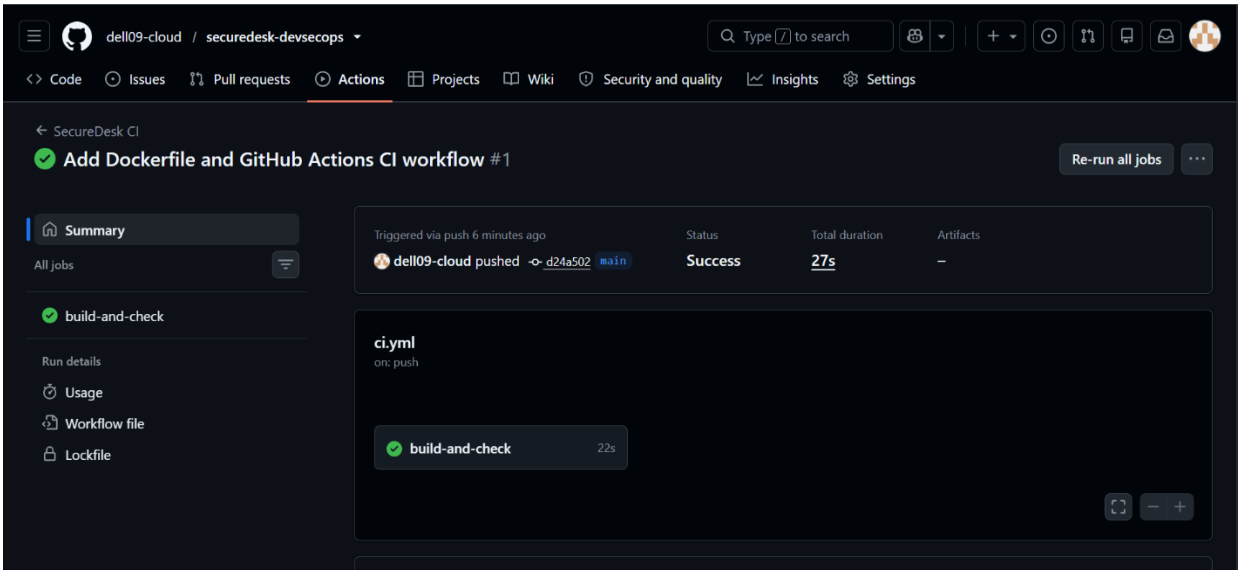
Created a Dockerfile for the SecureDesk Flask application using Python 3.14 as the base image. Installed the required Python dependencies and Gunicorn, copied the application source code into the image, and configured the container to run using a non-root user for improved security. The Docker image was successfully built, and a test container was started to verify the application. The container responded with HTTP 200 OK. This was performed as a separate Docker test; the live production application continues to run on the EC2 instance using Nginx, Gunicorn, Flask and the systemd service. The test confirmed successful container startup and application availability. Docker is used for build and test; production runs under systemd on EC2.



Successful Docker container execution and HTTP 200 OK response from the SecureDesk application.

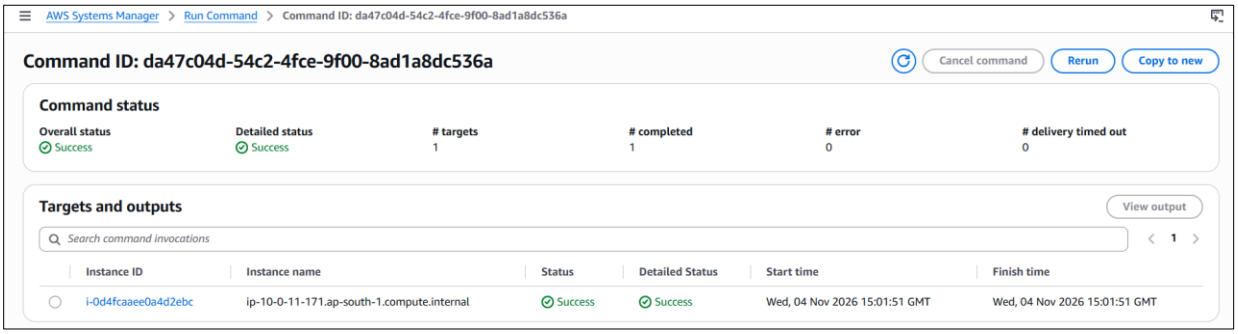
Step 2. GitHub actions CI pipeline

Configured a GitHub Actions workflow in the SecureDesk repository to automate continuous integration. The workflow is triggered whenever code is pushed to the main branch or a pull request is created targeting the main branch. The pipeline automatically checks out the source code, sets up Python 3.14, validates Python syntax, and builds the Docker image. Successfully executed the workflow and verified that all pipeline jobs completed with a green success status.



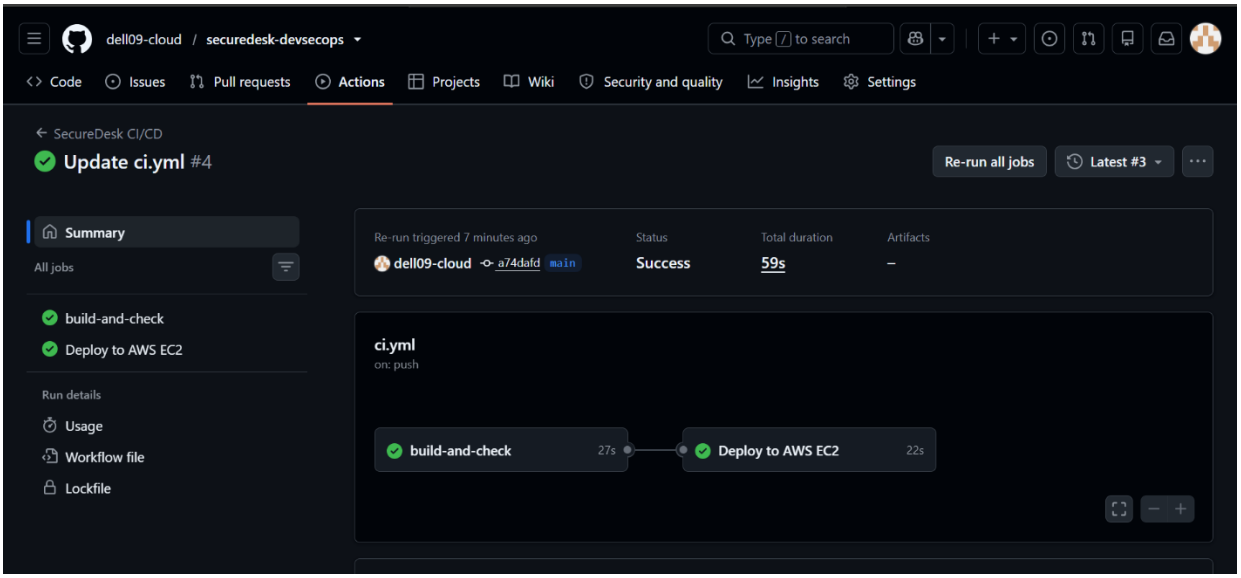
Step 3. Automated deployment

Configured GitHub Actions to authenticate securely with AWS using OpenID Connect (OIDC), eliminating the need to store long-term AWS access keys. Created an IAM deployment role with restricted permissions to access the private EC2 instance through AWS Systems Manager. Successfully verified the instance connection and executed remote commands to check Docker installation and the SecureDesk application service status.



Step 3.1. Automated Deployment Verification:

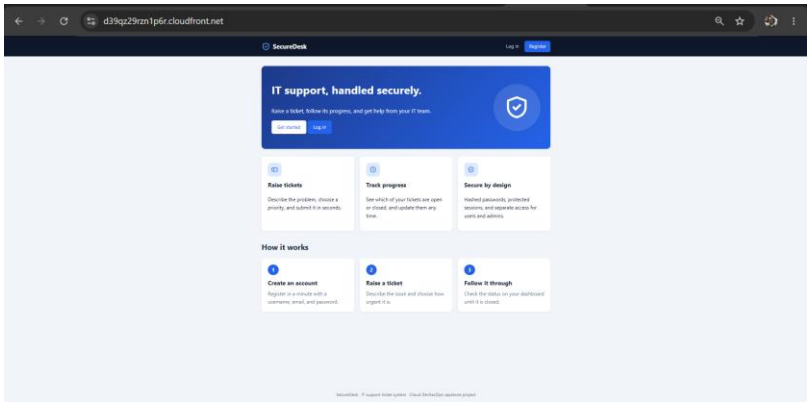
Successfully tested the automated deployment through GitHub Actions. The workflow completed successfully, confirming that the application code was pulled from GitHub, dependencies were installed, and the SecureDesk service was restarted on the private EC2 instance. Verified that the securedesk application service was active after deployment. AWS Systems Manager was used to execute deployment commands securely without requiring SSH access from GitHub Actions.



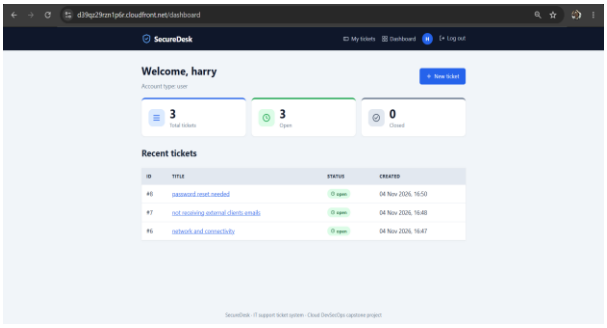
PHASE 10. Finalization

Step 1. Application Screenshots

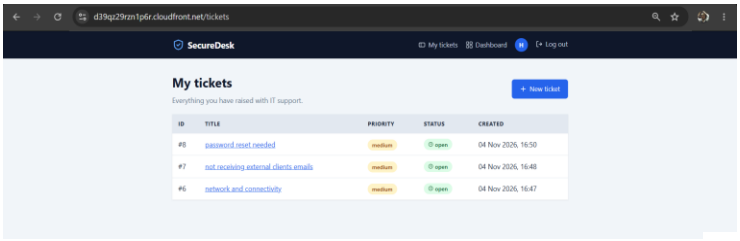
User and Admin pages



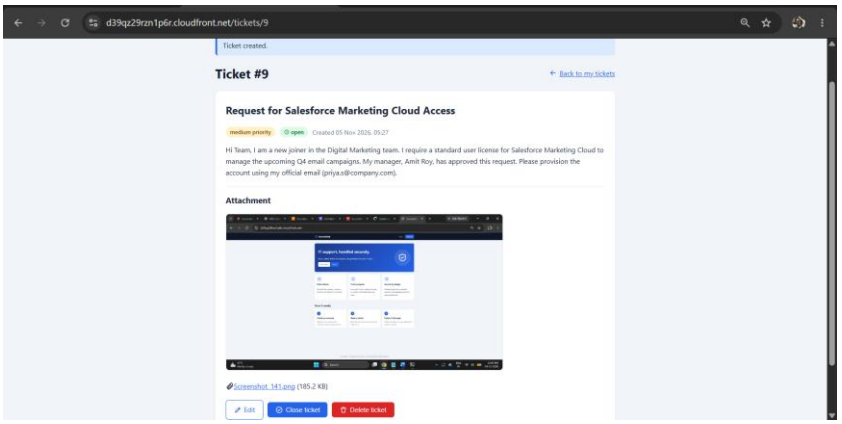
Home page of application before logged in



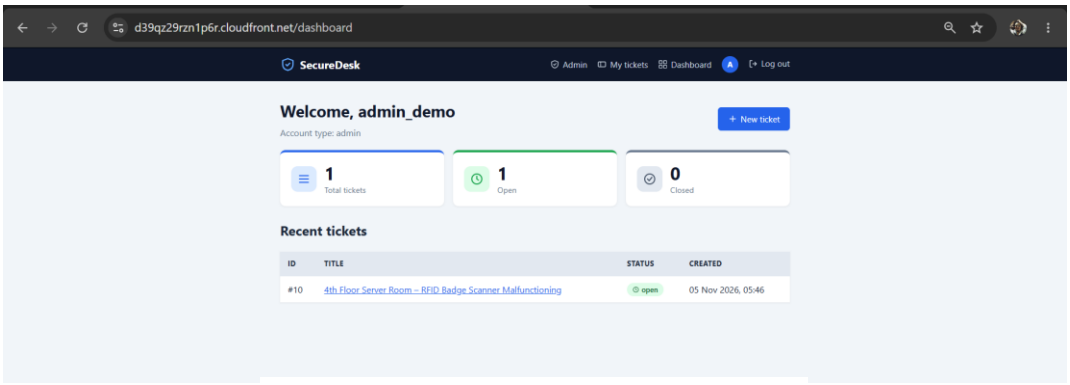
Dashboard of user



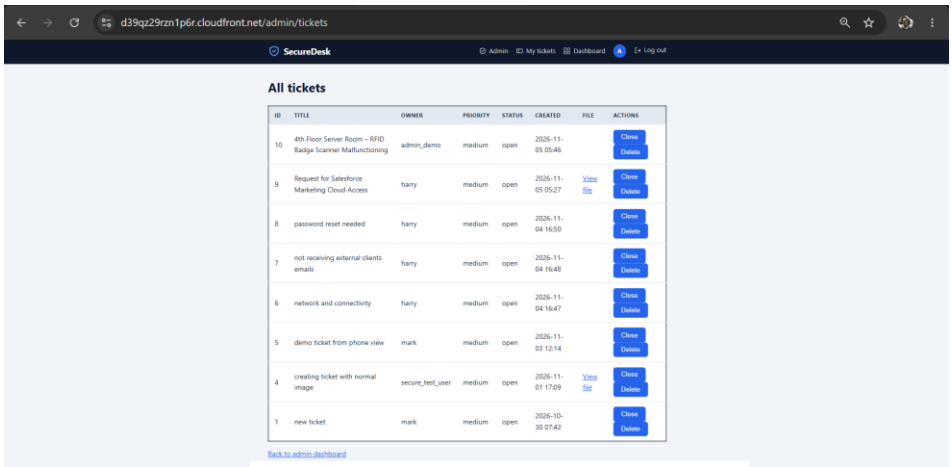
Ticket page of a user



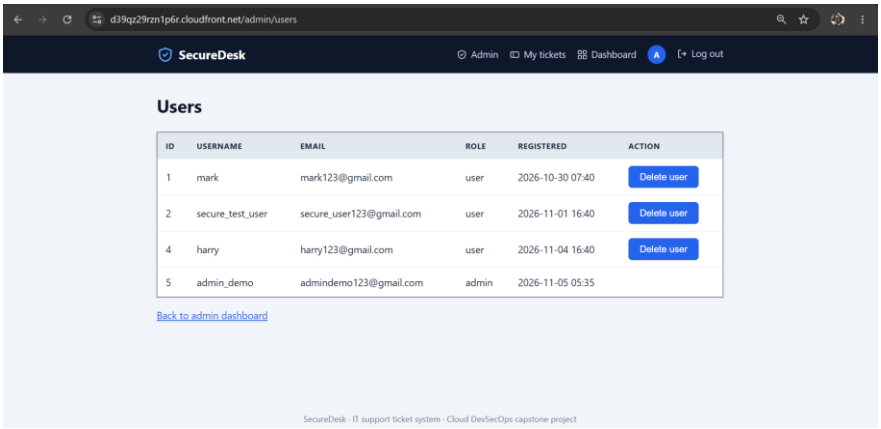
A ticket with an image attachment



Admin dashboard with a ticket



Admin all-tickets page



Users page of admin

Step 2. Security summary

This table lists each security control in the project and the phase where it's documented.

Control	How it's implemented	Where documented
Password hashing	Argon2 with a random salt	Phase 2, registration
SQL injection prevention	SQLAlchemy parameterized queries	Phase 2, registration
Input validation	Server-side checks on every form	Phase 2, registration and tickets
Sessions and cookies	HttpOnly, SameSite, Secure; one generic login error	Phase 2, login
CSRF protection	Hidden token on every form; POST for state changes	Phase 2, login and tickets
XSS protection	Output escaping, Content-Security-Policy	Phase 2, tickets and headers
Role-based access	Admin role checked on the server, 403 for others	Phase 2, admin
Rate limiting	Per IP and per username on login	Phase 2, rate limiting
Security headers and HSTS	Headers from the app, HSTS from CloudFront	Phase 2 headers, Phase 6
Secure file uploads	Type, content, size checks, random names, private bucket	Phase 4
Network isolation	Private subnets, least-privilege security groups	Phase 3
Encrypted, limited database access	TLS with certificate check, separate limited users	Phase 4, Phase 5
Secrets management	Root-only settings file, nothing	Phase 5

	in GitHub	
IAM roles, no stored keys	Instance role for S3; OIDC for the pipeline	Phase 4, Phase 9
Server hardening	SSH keys only, no root, fail2ban, UFW, bastion	Phase 5
HTTPS and edge protection	CloudFront HTTPS, firewall rules, load balancer locked to CloudFront	Phase 6
Backups	Restricted backup user	Phase 7
Monitoring and alerts	CloudWatch alarms and email alerts, tested	Phase 8
Container and pipeline security	Non-root container, keyless deployment	Phase 9

Conclusion

The SecureDesk project successfully demonstrates the deployment of a secure, cloud-based IT Support Ticket System using AWS and DevSecOps practices. It includes application hosting, database integration, security controls, backup and recovery, monitoring, and automated CI/CD deployment. This project provided valuable hands-on experience in cloud infrastructure management, application security, and DevSecOps workflows.

- end of report -